

Informe de Análisis y Corrección del MER

Sistema de Gestión de Convocatorias

Base de Datos · PostgreSQL

Fecha de generación: 19 de May de 2026

Errores detectados: 7 · Tablas nuevas: 4 · Mejoras aplicadas: 7

7

Errores detectados

4

Tablas nuevas/ajustadas

3

Objetos creados

1. Contexto del modelo analizado

El modelo entidad-relación (MER) analizado corresponde a un sistema de gestión de convocatorias académicas desarrollado en PostgreSQL. El archivo original (.pgerd) contenía cinco tablas principales: **convocatoria**, **usuario**, **programa**, **postulacion** y **evaluacion**, con sus respectivas relaciones de clave foránea.

Tras el análisis, se identificaron **7 problemas de diseño** que afectan la normalización (1FN, 2FN, 3FN), la integridad referencial, la trazabilidad y la escalabilidad del modelo. A continuación se detalla cada hallazgo junto con la corrección aplicada.

2. Tabla resumen de hallazgos

#	Tabla afectada	Problema	Solución aplicada	Impacto
1	usuario	rol como VARCHAR	Tabla rol + FK	3FN ✓
2	usuario	departamento/pais como texto plano	Tablas pais y departamento + FK	3FN ✓
3	postulacion	id_convocatoria redundante	FK eliminada; usar JOIN	Consistencia ✓
4	programa	Un solo coordinador (FK)	Tabla intermedia programa_usuario	Escalabilidad ✓
5	evaluacion	Sin id_evaluador	FK a usuario agregada	Trazabilidad ✓
6	Todas	Sin auditoría en varias tablas	fecha_registro + fecha_actualizacion	Auditoría ✓
7	Todas	Sin trigger de actualización	fn_actualizar_timestamp()	Automatización ✓

3. Errores detectados y correcciones aplicadas

Error #1 — Campo rol como VARCHAR en tabla usuario

La columna rol almacenaba directamente valores de texto ('aspirante', 'administrador') dentro de la tabla usuario. Esto viola la tercera forma normal (3FN) porque el significado del rol no depende de la clave primaria del usuario. Además, impide escalar permisos sin modificar la estructura de la tabla.

Antes (problema):

```
-- Antes CREATE TABLE usuario (
  id_usuario SERIAL PRIMARY KEY, rol
  VARCHAR(50) -- 'aspirante', 'admin' );
```

Después (corrección):

```
-- Después CREATE TABLE rol ( id_rol
  SERIAL PRIMARY KEY, nombre VARCHAR(50)
  NOT NULL UNIQUE ); CREATE TABLE usuario
  ( id_rol INT NOT NULL REFERENCES
  rol(id_rol) );
```

Error #2 — Columnas departamento y pais como texto libre en usuario

Los atributos departamento y pais se guardaban como VARCHAR directamente en usuario, lo que genera repetición masiva de datos (ej. 'Colombia' en miles de filas) y no permite validar que los valores sean correctos. Viola la 3FN por dependencia transitiva: departamento depende de pais, no de id_usuario.

Antes (problema):

```
-- Antes CREATE TABLE usuario (
  departamento VARCHAR(100), pais
  VARCHAR(100) DEFAULT 'Colombia' );
```

Después (corrección):

```
-- Después CREATE TABLE pais ( id_pais
  SERIAL PRIMARY KEY, nombre VARCHAR(100)
  UNIQUE ); CREATE TABLE departamento (
  id_departamento SERIAL PRIMARY KEY,
  id_pais INT REFERENCES pais ); --
  usuario.id_departamento INT REFERENCES
  departamento
```

Error #3 — FK redundante id_convocatoria en tabla postulacion

La tabla postulacion tenía claves foráneas hacia convocatoria Y hacia programa, pero programa ya referencia a convocatoria. Esto crea una dependencia transitiva: la convocatoria se puede obtener siempre a través del programa. Tener las dos FK duplica información y puede generar inconsistencias si se actualiza una pero no la otra.

Antes (problema):

```
-- Antes CREATE TABLE postulacion (
  id_convocatoria INT REFERENCES
  convocatoria, id_programa INT
  REFERENCES programa -- REDUNDANTE:
  programa ya -- referencia a
  convocatoria );
```

Después (corrección):

```
-- Después CREATE TABLE postulacion (
  id_programa INT REFERENCES programa --
  La convocatoria se obtiene via JOIN: --
  postulacion->programa->convocatoria );
```

Error #4 — FK directa id_usuario en programa (un solo coordinador)

La tabla programa tenía una única FK a usuario, lo que implica que solo un usuario puede coordinar un programa. En la realidad académica, un programa puede tener varios coordinadores, evaluadores o personal de apoyo. Esta restricción impide escalar el modelo sin romper la estructura.

Antes (problema):

```
-- Antes CREATE TABLE programa (
id_programa SERIAL PRIMARY KEY,
id_convocatoria INT REFERENCES
convocatoria, id_usuario INT REFERENCES
usuario -- solo 1 );
```

Después (corrección):

```
-- Después: tabla intermedia N:M CREATE
TABLE programa_usuario ( id_programa
INT REFERENCES programa, id_usuario INT
REFERENCES usuario, funcion
VARCHAR(50), -- coordinador/evaluador
UNIQUE(id_programa, id_usuario) );
```

Error #5 — Tabla evaluacion sin campo de evaluador

La tabla evaluacion registraba el resultado de una postulación pero no guardaba quién realizó la evaluación. Esto impide trazabilidad y auditoría del proceso: si hay un reclamo o inconsistencia, no se puede identificar qué usuario tomó la decisión. Incumple requisitos básicos de auditoría en sistemas académicos.

Antes (problema):

```
-- Antes CREATE TABLE evaluacion (
id_evaluacion SERIAL PRIMARY KEY,
id_postulacion INT REFERENCES
postulacion, resultado VARCHAR(30) --
No sabe quién evaluó );
```

Después (corrección):

```
-- Después CREATE TABLE evaluacion (
id_evaluacion SERIAL PRIMARY KEY,
id_postulacion INT REFERENCES
postulacion, id_evaluador INT
REFERENCES usuario, -- NUEVO resultado
VARCHAR(30), puntaje NUMERIC(5,2) );
```

Error #6 — Campos de auditoría ausentes en varias tablas

Solo la tabla usuario tenía campos fecha_registro y fecha_actualizacion. Las tablas convocatoria, programa, postulacion y evaluacion carecían de estos campos. Sin ellos es imposible rastrear cuándo se creó o modificó un registro, lo que es un requisito mínimo en cualquier sistema de información académica o administrativa.

Antes (problema):

```
-- Antes: solo en usuario CREATE TABLE
convocatoria ( id_convocatoria SERIAL,
titulo VARCHAR(200) -- Sin
fecha_registro ni --
fecha_actualizacion );
```

Después (corrección):

```
-- Después: en todas las tablas CREATE
TABLE convocatoria ( id_convocatoria
SERIAL PRIMARY KEY, titulo
VARCHAR(200), fecha_registro TIMESTAMP
DEFAULT NOW(), fecha_actualizacion
TIMESTAMP DEFAULT NOW() );
```

Error #7 — Tabla sin nombre (Untitled) en el diagrama

Una de las entidades del diagrama pgerd aparecía con el nombre 'Untitled', lo que indica que no fue renombrada correctamente en pgAdmin 4 antes de exportar. Una tabla sin nombre descriptivo no puede ser implementada ni referenciada correctamente en un esquema de producción.

Antes (problema):

```
-- En el .pgerd: "name": "Untitled" --  
La tabla no tiene nombre -- válido  
asignado
```

Después (corrección):

```
-- Corrección: asignar nombre -- según  
su responsabilidad, -- ej. si era tabla  
de soporte: CREATE TABLE  
documento_soporte ( id_documento SERIAL  
PRIMARY KEY, ... );
```

4. Nuevas tablas y objetos creados

+ Tabla pais

Centraliza los países del sistema. Incluye código ISO para validación estándar. Evita repetir texto libre como 'Colombia' en miles de registros.

```
CREATE TABLE pais ( id_pais SERIAL PRIMARY KEY, nombre VARCHAR(100) NOT NULL UNIQUE,
codigo_iso CHAR(2) NOT NULL UNIQUE );
```

+ Tabla departamento

Normaliza los departamentos geográficos con referencia al país. Permite validar que un departamento pertenezca a un país específico.

```
CREATE TABLE departamento ( id_departamento SERIAL PRIMARY KEY, nombre VARCHAR(100)
NOT NULL, id_pais INT NOT NULL REFERENCES pais, UNIQUE (nombre, id_pais) );
```

+ Tabla rol

Tabla catálogo para los roles del sistema (aspirante, administrador, evaluador). Permite agregar o modificar roles sin alterar la estructura de usuario.

```
CREATE TABLE rol ( id_rol SERIAL PRIMARY KEY, nombre VARCHAR(50) NOT NULL UNIQUE,
descripcion TEXT );
```

+ Tabla programa_usuario (relación N:M)

Reemplaza la FK directa id_usuario en programa. Permite que varios usuarios gestionen un mismo programa con diferentes funciones (coordinador, evaluador, apoyo).

```
CREATE TABLE programa_usuario ( id_programa INT REFERENCES programa, id_usuario INT
REFERENCES usuario, funcion VARCHAR(50) DEFAULT 'coordinador', UNIQUE (id_programa,
id_usuario) );
```

+ Función y triggers fn_actualizar_timestamp()

Función PostgreSQL reutilizable que actualiza automáticamente la columna fecha_actualizacion en cualquier tabla donde se aplique como trigger BEFORE UPDATE.

```
CREATE OR REPLACE FUNCTION fn_actualizar_timestamp() RETURNS TRIGGER AS $$ BEGIN
NEW.fecha_actualizacion = NOW(); RETURN NEW; END; $$ LANGUAGE plpgsql; -- Aplicado en:
usuario, convocatoria, programa, -- postulacion, evaluacion
```

+ Vista v_postulaciones_detalle

Vista que demuestra cómo obtener la convocatoria de una postulación sin FK redundante, usando JOIN transitivo: postulacion → programa → convocatoria.

```
CREATE VIEW v_postulaciones_detalle AS SELECT p.id_postulacion, u.nombre || ' ' ||  
u.apellido AS aspirante, pr.nombre AS programa, c.titulo AS convocatoria, p.estado  
FROM postulacion p JOIN usuario u ON u.id_usuario = p.id_usuario JOIN programa pr ON  
pr.id_programa = p.id_programa JOIN convocatoria c ON c.id_convocatoria =  
pr.id_convocatoria;
```

+ Índices de rendimiento

Se crearon 10 índices sobre columnas frecuentemente usadas en filtros y JOINS, asegurando que las consultas más comunes (buscar postulaciones por estado, buscar usuarios por email o cédula) sean eficientes.

```
CREATE INDEX idx_usuario_email ON usuario(email); CREATE INDEX idx_postulacion_estado  
ON postulacion(estado); CREATE INDEX idx_evaluacion_evaluador ON  
evaluacion(id_evaluador); -- + 7 índices adicionales
```

5. Esquema del modelo corregido

El siguiente diagrama muestra las relaciones entre las tablas del modelo corregido. Las tablas marcadas en verde son nuevas; las demás fueron modificadas.

Tabla	Tipo	Se relaciona con	Cambio principal
pais	Nueva ✓	departamento	Normaliza país
departamento	Nueva ✓	pais, usuario	Normaliza dpto.
rol	Nueva ✓	usuario	Roles escalables
usuario	Modificada	rol, departamento	FK en vez de texto
convocatoria	Modificada	usuario (creador)	Auditoría agregada
programa	Modificada	convocatoria	Sin FK directa a user
programa_usuario	Nueva ✓	programa, usuario	Relación N:M
postulacion	Modificada	programa, usuario	FK redundante eliminada
evaluacion	Modificada	postulacion, usuario	id_evaluador agregado

Tabla nueva

Tabla modificada

6. Conclusiones y recomendaciones

Normalización alcanzada: El modelo corregido cumple con la Primera, Segunda y Tercera Forma Normal (1FN, 2FN, 3FN). Ningún atributo no-clave depende transitivamente de otro atributo no-clave.

Integridad referencial: Todas las relaciones usan claves foráneas con reglas ON DELETE explícitas (RESTRICT, CASCADE o SET NULL), evitando registros huérfanos.

Escalabilidad: La tabla programa_usuario permite asignar múltiples roles a un programa sin modificar la estructura. La tabla rol permite agregar nuevos perfiles de usuario sin alterar usuario.

Trazabilidad y auditoría: Todos los cambios quedan registrados gracias a los campos fecha_registro y fecha_actualizacion presentes en cada tabla, actualizados automáticamente por triggers.

Rendimiento: Los 10 índices creados garantizan búsquedas eficientes sobre los campos más consultados: email, cédula, estado de postulación y evaluador.

El archivo **MER_corregido.sql** contiene el script completo listo para ejecutar en PostgreSQL 14+. Se recomienda ejecutarlo en un entorno de desarrollo antes de aplicarlo en producción.